GUÍA DE ESTUDIO — Modelo de Segundo Parcial

Técnicas de Programación · Python

> Guía de una sola lectura: el esqueleto común de los 3 ejercicios, las fórmulas, las trampas que se repiten y un checklist para rendir tranquilo.

---

1. EL ESQUELETO DE LOS 3 EJERCICIOS (todos son lo mismo)

```
DICCIONARIO GLOBAL  +  FUNCIONES QUE LEEN  +  UNA FUNCIÓN QUE ESCRIBE  +  main()
```

Ejercicio	Clave	Valor (qué guarda)
1. Estudiantes	nombre	lista de notas (float)
2. Stock	producto	lista de cantidades (int)
3. Inventario	producto	**diccionario** `{precio, stock, categoria}`

Regla de oro #1: solo **`agregar_*`** **muta** el diccionario, y **siempre al final**, después de validar TODO. Las demás funciones solo leen.

Regla de oro #2: si el programa tiene dos archivos, el diccionario se define **UNA sola vez** (en el archivo de funciones) y el principal lo **importa**: `from funciones import *`. Definirlo en dos archivos = el bug *"agregué y no existe"*.

---

2. PLANTILLA MAESTRA DE VALIDACIÓN (la que hay que tener memorizada)

```
def agregar_algo():
    # --- pedir hasta que sea válido ---
    while True:                        # bucle de reintento
        try:
            x = int(input("...: "))    # int() o float() según pida el enunciado
            if x <= 0:                 # o < 0 si el cero está permitido
                print("invalido")
                continue               # vuelve a pedir (no sale del while)
            break                      # válido: sale del while
        except ValueError:             # entrada no numérica
            print("Eso no es un número. Intente de nuevo.")

    # ...validar todo lo demás (nombre, existencia, etc.)...

    # --- LA MUTACIÓN VA AL FINAL ---
    mi_diccionario[nuevo_clave] = valor
```

- `while True` + `break`/`continue` -> repite hasta acertar.
- `try/except ValueError` -> captura `int("abc")` y `float("caro")`.
- **`strip()` SIEMPRE antes de `isalpha()`** -> `" Ana "` tiene espacios y fallaría.
- `nombre.isalpha()` -> solo letras (acepta tildes y ñ; rechaza números y espacios).
- Verificar `if clave in diccionario:` -> si ya existe, `return` **antes** de pedir más datos.

---

3. EJERCICIO 1 — Estudiantes y notas

Funciones (6):

1. `agregar_estudiante()` — valida nombre/cantidad/notas y guarda. Notas con **`float` > 0**.

2. `calcular_promedio(e, n)` -> `sum(notas) / len(notas)`; si no existe o la lista está vacía -> **imprime el motivo y retorna `None`**.
3. `obtener_maximo_promedio(e)` -> recorre guardando el máximo; retorna **tupla** `(nombre, promedio)`. **Obligatorio:** `if max_promedio is None or promedio > max_promedio` (en Python 3, `None > float` = `TypeError`).
4. `clasificar_estudiante(p)` -> **ORDEN IMPORTANTE:** `python if p >= 9: return "Excelente" # primero el umbral más alto if p >= 7: return "Aprobado" return "Reprobado" # si no, nunca nadie es "Excelente"`
5. `mostrar_resumen(e, n)` -> llama a 2 y a 4; si el promedio es `None`, hace `return` antes de formatear (`f"{None:.2f}"` = `TypeError`).
6. `obtener_aprobados(e)` -> comprensión usando `calcular_promedio()`: `python return [n for n in estudiantes if estudiantes[n] and calcular_promedio(estudiantes, n) >= 7]` (cortocircuito: lista vacía -> no llama, no imprime de más)

[!] ERRATA del enunciado: dice que da `["Ana", "Luis"]`, pero Luis promedia `(7.0 + 6.5 + 7.2) / 3 = 6.9` -> **< 7** -> el real es `["Ana"]`. **Confío en la regla, no en el ejemplo.** (Ana: 9.17 -> Excelente.)

---

4. EJERCICIO 2 — Productos y stock

Datos: igual al Ej 1 pero con enteros; `"Impresora": []` es el caso borde.

- **Punto 1 — `agregar_producto()`:** mismo esqueleto, pero cada registro es **`int` positivo** -> `int("3.5")` = error (a diferencia de las notas del Ej 1).
- **Punto 2 — elegir UNA:**
- **Opción A** `calcular_stock_promedio(p, n)` -> `sum / len` (igual que promedio del Ej 1).
- **Opción B** `sumar_stock_total(p, n)` -> solo `sum(registros)`.
- Las dos: `¿existe?` -> `¿lista vacía?` -> calcula. Si no -> `print` + `return None`.
- **Punto 3 — `obtener_productos_bajo_stock(p)`:** `python return [n for n in productos if productos[n] and calcular_stock_promedio(productos, n) < 5]`
- `< 5` **estricto** (4.99 sí, 5 no).
- Excluir vacías **sin mensajes** -> el guard `productos[n] and ...` va primero.
- Ejemplo del enunciado `["Monitor"]` -> **correcto** (verificado).
- Si elegiste la Opción B: calculá el promedio adentro (`sum(...)/len(...) < 5`).

---

5. EJERCICIO 3 — Inventario ANIDADO

Estructura = dos saltos de clave:

```
inventario["Mouse"]["precio"]        # 1er salto: producto -> 2do: campo
producto = inventario[nombre]        # atajo: tomar el diccionario interno una vez
producto["stock"]
```

- **Punto 1 — `agregar_producto()`:** 3 datos a pedir y validar:
- nombre: `strip()` + `isalpha()` + no repetido;
- **precio: `float` > 0** (acepta decimales);
- **stock: `int` >= 0** -> el **cero SÍ vale** -> chequeo `if stock < 0` (¡no `<= 0`!);
- categoría: texto no vacío (sin try/except, no hay conversión);
- guardar crea un **diccionario nuevo**: `inventario[nombre] = {"precio": p, "stock": s, "categoria": c}`
- **Punto 2 — `consultar_producto(inv, n)`:** `if n not in inventario` -> informa; si existe -> imprime precio, stock y categoría. Comillas: **simples adentro** del f-string: `f"{producto['precio']}"`.
- **Punto 3 — `calcular_valor_inventario(inv)`:** `python total = 0 for nombre, producto in inventario.items(): # .items() da (clave, valor) total += producto["precio"] * producto["stock"] return total # con los datos: 815000` Impresora (`stock 0`) aporta `0` -> no rompe (acá **no se divide** nada).
- **Punto 4 — `filtrar_por_categoria(inv, cat)`:** comprensión con `inventario[nombre]["categoria"] == categoria` -> **comparación exacta** (`"perifericos"` != `"Perifericos"` -> `[]`).
- **Punto 5 — `productos_sin_stock(inv)`:** comprensión con `["stock"] == 0` -> con los datos: `["Impresora"]` [OK] (ejemplo correcto).

6. LAS 7 TRAMPAS QUE SE REPITEN

1. **`strip()` antes de `isalpha()`** (sino el nombre con espacios no pasa).
2. **Caso borde = lista/diccionario vacío** -> siempre preguntarlo **antes** de dividir: `if not lista: print(...); return None` (evita `ZeroDivisionError`).
3. **`None` no se formatea** -> `f"{None:.2f}"` = `TypeError`. Siempre: `if resultado is None: return` **antes** de imprimir con formato.
4. **Orden de los `if` de clasificación** -> de mayor a menor (`>= 9` antes que `>= 7`).
5. **`float` vs `int` según el enunciado:** notas/precios -> `float`; cantidades/stock -> `int` (y `int("3.5")` falla a propósito).
6. **`> 0` vs `>= 0`:** notas/cantidad -> positivo estricto; stock inicial del Ej 3 -> **el cero vale**. Leé bien "mayor a cero" vs "mayor o igual a cero".
7. **Duplicar el diccionario en dos archivos** -> el alta va a un lado y la consulta al otro. Un solo diccionario + `from archivo import *`.

7. CONCEPTOS DE PYTHON QUE CAEN GARANTIZADOS

- **Variable = etiqueta**, no caja. `b = a` apunta al **mismo objeto** (no copia).
- **Scope local vs global:** las variables de una función mueren al hacer `return`; la global vive siempre. `mi_dic[x] = y` (asignación a un elemento) usa la global sin necesidad de `global`; `mi_dic = {...}` (reasignar el nombre) sí la crearía local.
- **`return` entrega / `print` muestra.** `None` = "no tengo resultado".
- **Comprensión de lista:** `[x for x in coleccion if condicion]` -> recorre, filtra y construye **una lista nueva** sin tocar el original.
- **Cortocircuito de `and`:** si la primera parte es falsa, la segunda **no se ejecuta** -> sirve como guard: `if lista and calculo(...) < 5`.
- **`.items()`** -> pares (clave, valor) para recorrer diccionarios.
- **Tupla + unpacking:** `nom, prom = obtener_maximo(...)`.
- **`if __name__ == "__main__": main()`** -> solo corre el menú si ejecutás el archivo; si lo importan, solo carga las funciones (contar **2 guiones bajos** en cada lado).
- **`from archivo import *`** comparte el **mismo objeto** (por eso arregla el bug de los diccionarios duplicados).

8. ERRORES TÍPICOS (los que salieron en la práctica) -> SOLUCIÓN

Error	Causa	Solución
`TypeError: list.append() takes exactly one argument (0 given)`	`notas.append()` sin argumento	`notas.append(nota)` — siempre llevar el valor
`NameError: name 'estudiantes' is not defined`	falta el diccionario arriba del archivo	pegarlo **antes** del primer `def`
"Agregué y luego dice que no existe"	**dos copias** del diccionario (uno por archivo)	una sola definición en el de funciones + importar en el principal
El programa corre pero **no muestra nada**	falta/guarda mal el `if __name__ == "__main__":` o `main` sin `()`	últimas dos líneas: guard con 2+2 guiones bajos y `main()` con paréntesis; **Ctrl+S**
`ZeroDivisionError`	promedio con lista vacía	chequear `if not lista` antes de dividir
`TypeError: '<' not supported between 'NoneType' and 'float'`	comparar el candidato inicial `None`	`if maximo is None or x > maximo:`
El menú no aparece al importar otro archivo	código suelto sin guard	envolver en `def main()` + guard

9. MÉTODO DE ATAQUE PARA EL PARCIAL (5 pasos)

1. **Identificar la estructura:** clave -> ¿lista? (Ej 1-2) o ¿dict anidado? (Ej 3).

2. **Listar los casos borde ANTES de codear:** clave inexistente, lista vacía, entrada no numérica, negativo/cero, repetido.
3. **Separar leer de escribir:** solo `agregar_*` muta, y al final.
4. **Escribir el código** con la plantilla de validación (sección 2).
5. **Probar con 1 caso normal + 1 caso borde** y verificar el ejemplo del enunciado (si no cierra, **vale la regla, no el ejemplo** — Ej 1).

10. CHECKLIST ANTES DE ENTREGAR

- [ ] El diccionario está **una sola vez**, arriba de todo del archivo de funciones.
- [ ] `strip()` antes de `isalpha()` en todos los altas.
- [ ] Todo `input` numérico dentro de `try/except ValueError`.
- [ ] La mutación del diccionario es la **última línea** de `agregar_*`.
- [ ] Los casos borde retornan `None` **avisando**, y el llamador chequea `is None` antes de formatear.
- [ ] Las comprensiones tienen el guard de lista vacía **primero**.
- [ ] `clasificar_estudiante` pregunta `>= 9` **antes** que `>= 7`.
- [ ] `int` donde pide enteros, `float` donde pide números; `> 0` vs `>= 0` correcto.
- [ ] `main()` termina con `if __name__ == "__main__":` + `main()` en la columna 0.
- [ ] **Ctrl+S** antes de correr.

Datos de prueba oficiales (para verificar rápido)

```
Ej 1: Ana 9.17 (Excelente) · Luis 6.90 (Reprobado) · Marta 6.0 · Carlos sin notas
      max_promedio = ("Ana", 9.1666…) · aprobados = ["Ana"]   <- errata del enunciado
Ej 2: Mouse 10 · Teclado 6 · Monitor 3 · Impresora sin registros
      bajo stock < 5 = ["Monitor"]                            <- ejemplo correcto
Ej 3: valor total = 815000 · sin stock = ["Impresora"]        <- ejemplo correcto
      Perifericos = ["Mouse", "Teclado"]
```

Archivos con las soluciones completas: `ejercicio1_estudiantes.py`, `ejercicio2_stock.py`, `ejercicio2_principal.py`, `ejercicio3_inventario.py`.

Anexo A — ejercicio1_estudiantes.py

```
"""
Ejercicio 1 - Modelo de Segundo Parcial
Tecnicas de Programacion - Python
Gestion de estudiantes y notas
"""

# ============================================================================
# Datos iniciales (entregados por el enunciado)
#   clave  -> nombre del estudiante (str)
#   valor -> lista de notas (list de float).  [] = todavia sin notas
# ============================================================================
estudiantes = {
    "Ana": [9.5, 9.2, 8.8],
    "Luis": [7.0, 6.5, 7.2],
    "Marta": [6.0, 5.8, 6.2],
    "Carlos": [],
}


# ----------------------------------------------------------------------------
# 1) Agregar estudiantes y notas
#     El enunciado la pide SIN parametros -> trabaja sobre la variable
#     global `estudiantes` (en la vida real la pasariamos como parametro).
# ----------------------------------------------------------------------------
def agregar_estudiante():
    """Pide los datos al usuario, los valida TODO y recien ahi muta el diccionario."""

    # --- a) Nombre: unicamente letras, sin espacios sobrantes --------------
    while True:
        nombre = input("Nombre del estudiante: ").strip()   # (1) saca espacios
        if nombre == "":                                    # (2) quedo vacio?
            print("  El nombre no puede quedar vacio.")
            continue
        if not nombre.isalpha():                            # (3) solo letras?
            print("  El nombre debe contener unicamente letras.")
            continue
        break                                               # (4) nombre valido

    # --- b) El estudiante no debe existir todavia --------------------------
    if nombre in estudiantes:                               # (5) ya existe?
        print(f"  '{nombre}' ya esta cargado; no se repite.")
        return                                              # (6) corta aca

    # --- c) Cuantas notas quiere cargar? -----------------------------------
    while True:
        try:
            cantidad = int(input("Cuantas notas quiere ingresar? "))  # (7)
            if cantidad <= 0:                               # (8) entero positivo?
                print("  Debe ser un entero mayor a cero.")
                continue
            break                                           # (9) cantidad valida
        except ValueError:                                  # (10) int() fallo
            print("  Eso no es un numero entero. Intente de nuevo.")

    # --- d) Cada nota: numero positivo -------------------------------------
    notas = []                                              # (11) lista nueva
    while len(notas) < cantidad:                            # (12) faltan notas?
        try:
            nota = float(input("  Nota %d: " % (len(notas) + 1)))  # (13)
            if nota <= 0:                                   # (14) positiva?
                print("  La nota debe ser un numero positivo.")
                continue
            notas.append(nota)                              # (15) acumula
        except ValueError:                                  # (16) float() fallo
            print("  Eso no es un numero. Intente de nuevo.")

    # --- e) Recien AHORA se toca el diccionario ----------------------------
    estudiantes[nombre] = notas                             # (17) mutacion final
    print(f"  Estudiante '{nombre}' agregado con {cantidad} nota(s).")


# ----------------------------------------------------------------------------
# 2) Calcular promedio
# ----------------------------------------------------------------------------
def calcular_promedio(estudiantes, nombre):
    """Promedio de las notas de `nombre`.
    Retorna None (y avisa) si el estudiante no existe o no tiene notas."""
```

```
    if nombre not in estudiantes:                           # (1) existe?
        print(f"  El estudiante '{nombre}' no existe.")
        return None                                         # (2) senal de fallo
    notas = estudiantes[nombre]                             # (3) referencia
    if not notas:                                           # (4) lista vacia?
        print(f"  '{nombre}' no tiene notas cargadas.")
        return None                                         # (5) senal de fallo
    return sum(notas) / len(notas)                          # (6) division real


# ---------------------------------------------------------------------------
# 3) Obtener el maximo promedio
# ---------------------------------------------------------------------------
def obtener_maximo_promedio(estudiantes):
    """Retorna una tupla (nombre, promedio) del mejor promedio."""
    max_nombre = None
    max_promedio = None
    for nombre in estudiantes:
        promedio = calcular_promedio(estudiantes, nombre)
        if promedio is None:                                # sin notas -> salta
            continue
        # "None o ..." protege la primera vuelta: en Python 3 NO se puede
        # comparar None con un numero (TypeError).
        if max_promedio is None or promedio > max_promedio:
            max_nombre = nombre
            max_promedio = promedio
    return max_nombre, max_promedio


# ---------------------------------------------------------------------------
# 4) Clasificar estudiante
#     OJO EL ORDEN: primero el umbral mas alto, si no nunca llegarias
#     a "Excelente" (Ana tambien cumple >= 7).
# ---------------------------------------------------------------------------
def clasificar_estudiante(promedio):
    if promedio >= 9:
        return "Excelente"
    if promedio >= 7:                   # aca ya sabemos que es < 9
        return "Aprobado"
    return "Reprobado"


# ---------------------------------------------------------------------------
# 5) Mostrar resumen (reutiliza 2 y 4)
# ---------------------------------------------------------------------------
def mostrar_resumen(estudiantes, nombre):
    promedio = calcular_promedio(estudiantes, nombre)
    if promedio is None:                # ya se informo el motivo arriba
        return
    clasificacion = clasificar_estudiante(promedio)
    print(f"  Estudiante   : {nombre}")
    print(f"  Promedio     : {promedio:.2f}")
    print(f"  Clasificacion: {clasificacion}")


# ---------------------------------------------------------------------------
# 6) Aprobados con LISTA POR COMPRENSION
#     Debe usar calcular_promedio() -> lo llamamos dentro del filtro.
#     `estudiantes[nombre] and ...` = cortocircuito: si la lista esta
#     vacia NO se llama a calcular_promedio (no calcula ni imprime de mas).
# ---------------------------------------------------------------------------
def obtener_aprobados(estudiantes):
    return [
        nombre
        for nombre in estudiantes
        if estudiantes[nombre] and calcular_promedio(estudiantes, nombre) >= 7
    ]


# ---------------------------------------------------------------------------
# Programa principal (menu minimo para que sea un "programa")
# ---------------------------------------------------------------------------
def main():
    while True:
        print("\n===== ESTUDIANTES =====")
        print("1) Agregar estudiante y notas")
        print("2) Calcular promedio")
        print("3) Resumen (promedio + clasificacion)")
        print("4) Estudiante con maximo promedio")
        print("5) Lista de aprobados")
```

```
        print("0) Salir")
        opcion = input("Opcion: ").strip()

        if opcion == "1":
            agregar_estudiante()
        elif opcion == "2":
            nom = input("Nombre: ").strip()
            prom = calcular_promedio(estudiantes, nom)
            if prom is not None:
                print(f"  Promedio de {nom}: {prom:.2f}")
        elif opcion == "3":
            mostrar_resumen(estudiantes, input("Nombre: ").strip())
        elif opcion == "4":
            nom, prom = obtener_maximo_promedio(estudiantes)
            if prom is None:            # ojo: None no admite :.2f
                print("  No hay estudiantes con notas.")
            else:
                print(f"  Maximo promedio: {nom} ({prom:.2f})")
        elif opcion == "5":
            print(f"  Aprobados: {obtener_aprobados(estudiantes)}")
        elif opcion == "0":
            print("Hasta luego!")
            break
        else:
            print("  Opcion invalida.")


if __name__ == "__main__":
    main()
```

Anexo B — ejercicio2_stock.py

```
"""
Ejercicio 2 - Modelo de Segundo Parcial
Tecnicas de Programacion - Python
Gestion de productos y stock
"""

# ============================================================================
# Datos iniciales (entregados por el enunciado)
#   clave  -> nombre del producto (str)
#   valor -> lista de cantidades registradas en controles de stock (int)
#            [] = todavia sin registros  (caso borde: "Impresora")
# ============================================================================
productos = {
    "Mouse": [10, 8, 12],
    "Teclado": [5, 7, 6],
    "Monitor": [3, 4, 2],
    "Impresora": [],
}


# ----------------------------------------------------------------------------
# 1) Agregar productos y cantidades  (mismo patron que el Ejercicio 1)
# ----------------------------------------------------------------------------
def agregar_producto():
    """Pide los datos, los valida TODO y recien ahi muta el diccionario."""

    # --- a) Nombre: unicamente letras, sin espacios sobrantes --------------
    while True:
        nombre = input("Nombre del producto: ").strip()      # (1)
        if nombre == "":                                     # (2)
            print("  El nombre no puede quedar vacio.")
            continue
        if not nombre.isalpha():                             # (3)
            print("  El nombre debe contener unicamente letras.")
            continue
        break                                                # (4) nombre valido

    # --- b) El producto no debe existir todavia ----------------------------
    if nombre in productos:                                  # (5)
        print(f"  '{nombre}' ya esta cargado; no se repite.")
        return                                               # (6)

    # --- c) Cuantas cantidades quiere registrar? ---------------------------
    while True:
        try:
            cantidad = int(input("Cuantas cantidades desea registrar? "))  # (7)
            if cantidad <= 0:                                # (8)
                print("  Debe ser un entero mayor a cero.")
                continue
            break                                            # (9)
        except ValueError:                                   # (10)
            print("  Eso no es un numero entero. Intente de nuevo.")

    # --- d) Cada cantidad: numero ENTERO positivo --------------------------
    registros = []                                           # (11)
    while len(registros) < cantidad:                         # (12)
        try:
            cantidad_stock = int(input("  Cantidad %d: " % (len(registros) + 1)))  # (13)
            if cantidad_stock <= 0:                          # (14)
                print("  La cantidad debe ser un entero positivo.")
                continue
            registros.append(cantidad_stock)                 # (15)
        except ValueError:                                   # (16)
            print("  Eso no es un entero. Intente de nuevo.")

    # --- e) Recien AHORA se toca el diccionario ----------------------------
    productos[nombre] = registros                            # (17)
    print(f"  Producto '{nombre}' agregado con {cantidad} registro(s).")


# ----------------------------------------------------------------------------
# 2) Opcion A: Calcular stock promedio  (la que elegi desarrollar)
# ----------------------------------------------------------------------------
def calcular_stock_promedio(productos, nombre):
    """Promedio de las cantidades registradas de `nombre`.
    Retorna None (y avisa) si no existe o no tiene registros."""
    if nombre not in productos:                              # (1) existe?
```

```
        print(f"  El producto '{nombre}' no existe.")
        return None                                          # (2)
    registros = productos[nombre]                            # (3)
    if not registros:                                        # (4) lista vacia?
        print(f"  '{nombre}' no posee registros de stock.")
        return None                                          # (5)
    return sum(registros) / len(registros)                   # (6)


# ---------------------------------------------------------------------------
# 2) Opcion B: Sumar stock total
#   (el parcial pide elegir UNA opcion; esta es practicamente identica
#    a la A, cambia solo la ultima linea y el mensaje)
# ---------------------------------------------------------------------------
def sumar_stock_total(productos, nombre):
    """Suma de las cantidades registradas de `nombre`."""
    if nombre not in productos:
        print(f"  El producto '{nombre}' no existe.")
        return None
    registros = productos[nombre]
    if not registros:
        print(f"  '{nombre}' no posee registros de stock.")
        return None
    return sum(registros)


# ---------------------------------------------------------------------------
# 3) Productos con stock promedio menor a 5 (LISTA POR COMPRENSION)
# ---------------------------------------------------------------------------
def obtener_productos_bajo_stock(productos):
    """Nombres de productos con promedio < 5, excluyendo los sin registros."""
    return [
        nombre
        for nombre in productos
        # cortocircuito: si la lista esta vacia NO se llama a
        # calcular_stock_promedio (no calcula ni imprime de mas)
        if productos[nombre] and calcular_stock_promedio(productos, nombre) < 5
    ]


# ---------------------------------------------------------------------------
# Programa principal (menu)
# ---------------------------------------------------------------------------
def main():
    while True:
        print("\n===== PRODUCTOS Y STOCK =====")
        print("1) Agregar producto y cantidades")
        print("2) Stock promedio de un producto (Opcion A)")
        print("3) Suma total de stock de un producto (Opcion B)")
        print("4) Productos con stock promedio menor a 5")
        print("0) Salir")
        opcion = input("Opcion: ").strip()

        if opcion == "1":
            agregar_producto()
        elif opcion == "2":
            nom = input("Producto: ").strip()
            prom = calcular_stock_promedio(productos, nom)
            if prom is not None:                 # None no admite :.2f
                print(f"  Stock promedio de {nom}: {prom:.2f}")
        elif opcion == "3":
            nom = input("Producto: ").strip()
            total = sumar_stock_total(productos, nom)
            if total is not None:
                print(f"  Stock total de {nom}: {total}")
        elif opcion == "4":
            print(f"  Bajo stock: {obtener_productos_bajo_stock(productos)}")
        elif opcion == "0":
            print("Hasta luego!")
            break
        else:
            print("  Opcion invalida.")


if __name__ == "__main__":
    main()
```

Anexo C — ejercicio3_inventario.py

```
"""
Ejercicio 3 - Modelo de Segundo Parcial
Tecnicas de Programacion - Python
Gestion de productos y precios  ->  DICCIONARIO ANIDADO
"""

# ============================================================================
# Datos iniciales (entregados por el enunciado)
#   clave  -> nombre del producto (str)
#   valor -> OTRO diccionario con tres campos:
#              "precio"    -> numero positivo
#              "stock"     -> entero >= 0  (el cero SI se permite)
#              "categoria" -> texto
# ============================================================================
inventario = {
    "Mouse": {"precio": 15000, "stock": 10, "categoria": "Perifericos"},
    "Teclado": {"precio": 25000, "stock": 5, "categoria": "Perifericos"},
    "Monitor": {"precio": 180000, "stock": 3, "categoria": "Pantallas"},
    "Impresora": {"precio": 95000, "stock": 0, "categoria": "Impresion"},
}


# ----------------------------------------------------------------------------
# 1) Agregar productos
# ----------------------------------------------------------------------------
def agregar_producto():
    """Pide nombre, precio, stock y categoria; valida TODO;
    recien ahi crea un diccionario nuevo dentro del inventario."""

    # --- a) Nombre: unicamente letras, sin espacios sobrantes --------------
    while True:
        nombre = input("Nombre del producto: ").strip()      # (1)
        if nombre == "":                                     # (2)
            print("  El nombre no puede quedar vacio.")
            continue
        if not nombre.isalpha():                             # (3)
            print("  El nombre debe contener unicamente letras.")
            continue
        break                                                # (4)

    # --- b) No debe existir todavia ----------------------------------------
    if nombre in inventario:                                 # (5)
        print(f"  '{nombre}' ya esta cargado; no se repite.")
        return                                               # (6)

    # --- c) Precio: numero POSITIVO (acepta decimales) ---------------------
    while True:
        try:
            precio = float(input("Precio: "))                # (7)
            if precio <= 0:                                  # (8)
                print("  El precio debe ser un numero positivo.")
                continue
            break
        except ValueError:                                   # (9)
            print("  Eso no es un numero. Intente de nuevo.")

    # --- d) Stock: entero MAYOR O IGUAL A cero (el 0 SI vale) --------------
    while True:
        try:
            stock = int(input("Stock inicial: "))            # (10)
            if stock < 0:                                    # (11) ojo: < 0, no <= 0
                print("  El stock debe ser un entero mayor o igual a cero.")
                continue
            break
        except ValueError:                                   # (12)
            print("  Eso no es un numero entero. Intente de nuevo.")

    # --- e) Categoria: texto no vacio --------------------------------------
    while True:
        categoria = input("Categoria: ").strip()             # (13)
        if categoria == "":                                  # (14)
            print("  La categoria no puede quedar vacia.")
            continue
        break

    # --- f) Recien AHORA se muta el inventario -----------------------------
    #     Se CREA un diccionario nuevo con los tres campos y se asigna
```

```
    #     como valor de la clave `nombre`.
    inventario[nombre] = {                                  # (15)
        "precio": precio,                                   # (16)
        "stock": stock,                                     # (17)
        "categoria": categoria,                             # (18)
    }                                                       # (19)
    print(f"  Producto '{nombre}' agregado al inventario.")


# ---------------------------------------------------------------------------
# 2) Consultar producto
# ---------------------------------------------------------------------------
def consultar_producto(inventario, nombre):
    """Muestra precio, stock y categoria. Avisa si no existe."""
    if nombre not in inventario:                            # (1) existe?
        print(f"  El producto '{nombre}' no existe.")
        return                                              # (2)
    producto = inventario[nombre]                           # (3) diccionario INTERNO
    print(f"  Producto  : {nombre}")
    print(f"  Precio    : {producto['precio']}")            # (4) dos saltos de clave
    print(f"  Stock     : {producto['stock']}")
    print(f"  Categoria : {producto['categoria']}")


# ---------------------------------------------------------------------------
# 3) Valor total del inventario  (precio x stock, sumado)
# ---------------------------------------------------------------------------
def calcular_valor_inventario(inventario):
    total = 0                                               # (1) acumulador
    for nombre, producto in inventario.items():             # (2) clave, valor
        valor = producto["precio"] * producto["stock"]      # (3) precio x stock
        total = total + valor                               # (4) suma al total
    return total                                            # (5)


# ---------------------------------------------------------------------------
# 4) Filtrar productos por categoria (LISTA POR COMPRENSION)
# ---------------------------------------------------------------------------
def filtrar_por_categoria(inventario, categoria):
    """Nombres de los productos cuya categoria coincida exactamente."""
    return [
        nombre
        for nombre in inventario
        if inventario[nombre]["categoria"] == categoria     # exacto, sin ignorar mayusculas
    ]


# ---------------------------------------------------------------------------
# 5) Productos sin stock (LISTA POR COMPRENSION)
# ---------------------------------------------------------------------------
def productos_sin_stock(inventario):
    """Nombres de los productos cuyo stock sea exactamente 0."""
    return [
        nombre
        for nombre in inventario
        if inventario[nombre]["stock"] == 0                 # cero, no negativo
    ]


# ---------------------------------------------------------------------------
# Programa principal (menu)
# ---------------------------------------------------------------------------
def main():
    while True:
        print("\n===== INVENTARIO =====")
        print("1) Agregar producto")
        print("2) Consultar producto")
        print("3) Valor total del inventario")
        print("4) Filtrar por categoria")
        print("5) Productos sin stock")
        print("0) Salir")
        opcion = input("Opcion: ").strip()

        if opcion == "1":
            agregar_producto()
        elif opcion == "2":
            consultar_producto(inventario, input("Producto: ").strip())
        elif opcion == "3":
            print(f"  Valor total: {calcular_valor_inventario(inventario)}")
        elif opcion == "4":
```

```
            cat = input("Categoria: ").strip()
            print(f"  Productos: {filtrar_por_categoria(inventario, cat)}")
        elif opcion == "5":
            print(f"  Sin stock: {productos_sin_stock(inventario)}")
        elif opcion == "0":
            print("Hasta luego!")
            break
        else:
            print("  Opcion invalida.")


if __name__ == "__main__":
    main()
```